Introduction to Principal Component Analysis (PCA) 📊

PCA is a popular **unsupervised learning algorithm** used primarily for **data visualization** and **dimensionality reduction**.

The Problem: High-Dimensional Data

Imagine you have a dataset with many features (e.g., 50 features describing countries, or 1000 features describing images).

- **Visualization Challenge:** You cannot plot a graph with 50 axes. Humans can only visualize 2D or 3D plots.
- **Redundancy:** Many features might be correlated (e.g., "Length of car" and "Width of car" often increase together).

The Solution: PCA

PCA allows you to take a dataset with many features (e.g., 50) and compress it down to just **2 or 3 new features** (often called **Principal Components**). This allows you to plot the data on a 2D graph to see patterns, clusters, or outliers.

How PCA Works (Conceptual Examples)

Example 1: Reducing 2 Features to 1

- **Data:** Car features. x_1 = Length, x_2 = Width.
- **Observation:** Most cars have a similar width (constrained by road lanes), but length varies a lot.
- **PCA Action:** PCA might decide that x_2 (Width) has little information (low variance) and project all data onto the x_1 axis, effectively keeping only the "Length" information.

Example 2: Correlated Features

- **Data:** Car features. x_1 = Length, x_2 = Height.
- **Observation:** Larger cars tend to be both longer *and* taller. The data points form a diagonal cloud.
- **PCA Action:** Instead of picking just Length or just Height, PCA creates a **new axis** (let's call it z) that runs diagonally through the data cloud.
    - This new z-axis represents the "Size" of the car (a combination of length and height).
    - By projecting data onto this new axis, we capture most of the information with a single number.

[Image of k-means clustering plot]

(Note: While the image above is for clustering, imagine a similar scatter plot where PCA finds the "best fit" line through the data to serve as a new axis.)

Example 3: Reducing 3D to 2D

- **Data:** A 3D cloud of points that looks like a flat pancake floating in space.
- **PCA Action:** PCA identifies the flat surface the data lies on. It creates two new axes (z_1, z_2) that span this "pancake." By re-mapping the data to these new axes, you can flatten the 3D data into a clear 2D plot without losing much information.

Real-World Application: Visualizing GDP

Imagine a dataset of countries with 50 economic features (x_1 = Total GDP, x_2 = GDP per capita, x_3 = Human Development Index, etc.).

- **Goal:** Visualize how countries compare to each other.
- **PCA Result:** PCA reduces these 50 features to 2 principal components (z_1, z_2).
  - z_1 **(Component 1):** Might roughly correspond to the "Overall Size" of the country's economy.
  - z_2 **(Component 2):** Might roughly correspond to the "Per-Person Wealth."
- **The Plot:** You can now plot every country on a simple 2D graph.
  - **USA:** High z_1 (Big economy), High z_2 (Rich citizens).
  - **Singapore:** Low z_1 (Small economy), High z_2 (Rich citizens).
  - This visualization helps you instantly understand the structure of complex, high-dimensional data.

1. Preprocessing: Normalization

Before the algorithm starts, you must normalize the data.

- **Zero Mean:** You subtract the mean from each feature so the data is centered at the origin (0,0).
- **Feature Scaling:** If x_1 is a huge number (like house size in square feet) and x_2 is a small number (like number of bedrooms), one feature will dominate the other. You scale them so they have a similar range.

2. Finding the New Axis (The Principal Component)

The goal is to find a new axis (let's call it the **z-axis**) that best captures the information in the data.

- **Projection:** PCA tests different directions for this new axis. For every data point, it draws a line perpendicular (at 90 degrees) to the new axis. This is called **projecting** the data.
- **Maximizing Variance:** How does it choose the best axis? It looks for the direction that keeps the data points **spread out** as much as possible.
    - **Bad Choice:** If the projected points are squished together, you have lost information (low variance).
    - **Good Choice:** If the projected points are widely spread out, you have retained the maximum amount of information (high variance).

This best axis is called the **Principal Component**.

3. The Math: Dot Products

Once PCA finds the direction of the z-axis, it defines it using a "unit vector" (a vector of length 1). Let's say this vector is $[0.71, 0.71]$.

To convert a data point (e.g., $x_1 = 2, x_2 = 3$) into the new feature z:
$z = \text{Vector}_{data} \cdot \text{Vector}_{axis}$
$z = [2, 3] \cdot [0.71, 0.71] = 3.55$

Your new feature z is **3.55**. You have successfully reduced 2 dimensions into 1.

4. What if you want more dimensions?

If you have 50 features and want to reduce them to 2 (instead of 1), PCA finds the second axis (z_2) by looking for a direction that is:

1. **Perpendicular (90 degrees)** to the first axis (z_1).
2. Captures the **next highest** amount of variance.

5. PCA is NOT Linear Regression

It is easy to confuse the two, but they do completely different things:

Feature	Linear Regression	PCA
Type	Supervised Learning (uses labels y)	Unsupervised Learning (no labels)
Goal	Predict y given x	Reduce features / Visualization
Error Calculation	Minimizes **Vertical** distance (squared error between point and line)	Minimizes **Perpendicular** distance (distance from point to the axis)

6. Reconstruction

You can also go backward. If you have the value $z = 3.55$, you can try to recreate the original x_1 and x_2.

- **Formula:** $z \times \text{Vector}_{axis}$
- **Result:** $3.55 \times [0.71, 0.71] = [2.52, 2.52]$

Implementing PCA with Scikit-learn 🖥️

Scikit-learn makes applying PCA straightforward. Here are the key steps and code examples.

1. Pre-processing: Feature Scaling

Before running PCA, it is critical to ensure all features are on a comparable scale.

- **Why?** If feature x_1 ranges from 0-1000 and x_2 ranges from 0-1, PCA will be biased toward x_1.
- **Action:** Standardize your data (e.g., using `StandardScaler` in scikit-learn) so each feature has a mean of 0 and variance of 1.

2. Fitting the Data

You use the `.fit()` method to calculate the new axes (principal components).

- **Note:** The `.fit()` function in PCA automatically performs mean normalization (centering the data), so you don't need to do that step manually (though feature scaling is still needed).

3. Inspecting Variance

After fitting, check how much information is retained using `.explained_variance_ratio_`.

- **Example:** `[0.992]` means the first component captures 99.2% of the information.
- This helps you decide if reducing dimensions (e.g., 2D to 1D) loses too much data.

4. Transforming the Data

Finally, use `.transform()` to project your original data onto the new axes. This gives you the new, compressed dataset.

Code Example

```
from sklearn.decomposition import PCA

# X is your dataset (e.g., 6 examples with 2 features)

# 1. Initialize PCA to find 1 principal component
pca_1 = PCA(n_components=1)

# 2. Fit the model to the data
pca_1.fit(X)

# 3. Check Explained Variance
```

```
# Result might be [0.992], meaning 99.2% of variance is captured
print(pca_1.explained_variance_ratio_)

# 4. Transform data to new dimensions
X_trans = pca_1.transform(X)
# X_trans is now an array of single numbers (1D data)
```

Advice on Applying PCA 💡

Primary Use Case: Visualization

- **Status: Highly Recommended / Common**
- **Purpose:** Taking high-dimensional data (50+ features) and reducing it to 2 or 3 dimensions to plot it. This helps you spot clusters, outliers, and patterns.

Secondary Use Case: Data Compression

- **Status:** *Less Common Today*
- **Purpose:** Reducing storage space or transmission bandwidth (e.g., compressing 50 features to 10).
- **Why Less Common?** Modern storage and bandwidth are cheap and plentiful, so aggressive compression is rarely necessary for structured data.

Secondary Use Case: Speeding Up Training

- **Status:** *Less Common Today*
- **Purpose:** Reducing features (e.g., 1000 $\rightarrow$ 100) to make supervised learning algorithms run faster.
- **Why Less Common?** Modern algorithms (like Neural Networks) handle high-dimensional data very well. The computational cost of running PCA often outweighs the small speedup in training.

Summary of Course 3, Week 2

This concludes the week on Recommender Systems and Dimensionality Reduction!

- **Collaborative Filtering:** Recommendations based on user behavior.
- **Content-Based Filtering:** Recommendations based on user/item features.
- **PCA:** Reducing dimensions to visualize complex data.